

Forge over Infinite Ordered State Spaces

Exact unsafe-region certificates for resources and lossy queues

A non-duplicating extension to the reviewed Forge design

Research and prototype prepared for Vladimir Reshetnikov

15 September 2026

Repository snapshots and limitations are specified in Section 1

Abstract

This article proposes two specialized workers for Forge: one for guarded nonnegative-affine resource transitions, including ordinary token consumption, reset, transfer and duplication; the other for finite-control lossy FIFO channels. Both operate over genuinely infinite state spaces. Their result is not merely a verdict about one initial state, but an exact finite description of *every* initial state from which a finite bad execution is possible.

The mathematical foundation is classical backward coverability in well-quasi-ordered systems. The contribution is a concrete, independently checkable implementation design for Forge: an immutable derivation DAG supplies one inclusion, and complete predecessor-coverage obligations supply the other. For affine resources, a proved clipping bound reduces infinite predecessor sets to integer boxes. For lossy queues, exact word-predecessor rules preserve FIFO order while accounting explicitly for message loss. A second producer checks closure directly against the current global antichain, avoiding the construction of large, irrelevant intermediate Pareto bases.

The accompanying standard-library Python implementation was executed. Its first producer emitted 190 accepted region certificates. Local differential tests made 63,504 resource-point comparisons and 2,325 word-pair comparisons. Complete finite execution oracles agreed on 7,000 resource queries, 2,040 acyclic-channel queries and 2,480 length-nonincreasing channel queries. A separation-based producer returned the identical bases on the same 190 models. On three deliberately constructed dense-preimage cases it completed where the eager producer reached its stated grid limit. One of these avoids explicitly listing 26,075,972,546 local minima; its full compact-JSON certificate is 1,063 bytes.

These are Python experiments, not Lean-kernel proofs. No Lean executable was available for this work, no Lean tactic was implemented, and no comparison against `grind` was run. The article specifies the missing Lean theorems, source-model bridges, integration points and acceptance tests rather than presenting that work as complete.

Contents

1	The extension boundary: what this adds, and what it does not	4
1.1	Reviewed snapshots	4
1.2	A narrowly defined capability delta	4
1.3	Relationship to established research	5
2	One exact question over two infinite domains	5
2.1	Unsafe means existential finite reachability	5
2.2	Order and upward bases	5
2.3	Why upward targets are an explicit restriction	6
3	A finite certificate for an exact infinite region	6
3.1	Two inclusions, with different evidence	6
3.2	The eager backward algorithm	7
3.3	Minimality and finite witness plans	8
4	Worker I: guarded affine resources	8
4.1	The accepted transition language	8
4.2	Reducing an infinite preimage to a finite integer box	9
4.3	A cheap exact rule for diagonal maps	10
4.4	Independent finite-box replay	10
4.5	Worked example: all semaphore populations at once	10
4.6	Merging and amplification	11
5	Worker II: lossy FIFO channels	11
5.1	The order is subsequence, not prefix or substring	11
5.2	Explicit loss semantics	12
5.3	Exact principal predecessors	12
5.4	A send loop and two ordered receives	12
5.5	Lossy is not reliable	13
6	Do not enumerate a local basis just to prove it is covered	13
6.1	The avoidable intermediate object	13
6.2	Direct global-region partition receipts	13
6.3	Closure separation as a search primitive	14
6.4	Executed ablation, with the exact claim	15
6.5	Further concrete optimizations, not yet implemented	15
7	What Forge should export after checking	16
7.1	An exact weakest safety precondition	16
7.2	Executable witnesses, not just basis membership	16

8	A concrete Lean integration plan	16
8.1	Keep the trust target small and semantic	17
8.2	A theorem dependency graph	17
8.3	Source-model bridges: three different permission levels	18
8.4	An implementable resource reifier	18
8.5	An implementable queue reifier	19
8.6	Concrete libraries and their roles	19
9	Executed experiments and what they establish	20
9.1	Environment and reproducibility	20
9.2	Local predecessor tests	21
9.3	Why the finite execution oracles really terminate	21
9.4	Corruption and refusal controls	21
9.5	A preserved failed run	22
9.6	Unmeasured claims	22
10	Implementation priorities and acceptance gates	22
10.1	First merge: a bounded, explicitly invoked worker	22
10.2	Required regression tests for the Lean port	22
10.3	Boundaries that should remain visible	23
11	Conclusion	23
A	Certificate format and executable entry points	23
A.1	Region object	23
A.2	Local receipt tags	24
A.3	Commands	24
B	Evidence inventory	25

1 The extension boundary: what this adds, and what it does not

1.1 Reviewed snapshots

The primary reference is Forge at commit

58ea206bd7ad501b930add568151217bcc7f82a2.

The review used its repository tree, root README, implementation-status inventory, Leant/Djex integration review, and merged finite-closure section. The auxiliary projects were inspected at Leant 6bf05ad78c467989e68290f2d08bbbed40802d485 and Djex e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5; the current indexing diagnostics were also available in their commit records. The precise URLs and audit scope are retained in docs/novelty-audit.md and docs/source-manifest.json.[1, 2, 3, 5, 6]

This was *not* a successful full local clone followed by an exhaustive review of every archived submission. Network access prevented the clone; connector reads provided the cited source material. A GitHub search returned an incomplete-results flag, so its zero matches were not used as proof of absence. Accordingly, “new” below means a concrete capability not present in the reviewed merged capability inventory and closure discussion, not a claim of priority over all literature or every archived file.

The distinction matters because the current Forge repository already contains three rounds of substantial work. It describes structural proof planning, observable-space and polynomial-ideal closure, finite datatype covers, noncommutative certificates, analytic estimates, quantitative probabilistic reasoning, and pushdown summaries. It also already establishes the intended separation between a producer and an acceptance boundary. Those ideas are prerequisites here, not new contributions.[2, 3]

1.2 A narrowly defined capability delta

Reviewed Forge capability	Reused, not reproposed	Added here
Linear and polynomial invariant closure	Target-directed finite certificates	Exact backward unsafe sets for ordered integer and word states
Finite-state and datatype covers	Finite records describing unbounded behavior	Dickson/Higman termination with unbounded counters and FIFO lengths
Pushdown summaries	Finite-control reachability vocabulary	FIFO channels with losses, not LIFO stacks
Polynomial cone and box certificates	Arithmetic leaves and coverage discipline	Nonnegative-affine integer preimage clipping and direct global coverage
Counterexample records and witness synthesis	Explicitly replayable evidence	A finite family of action words covering <i>all</i> unsafe initial states
Leant/Djex evidence boundaries	Exact subjects and approximation polarity	Model-specific simulation obligations for counters, queues and population quotients

Table 1: The contribution is two executable semantic workers and their particular certificates, not another generic controller.

A semaphore theorem can be proved by a familiar hand-written invariant. That is not the strongest reason to add this worker. The stronger reason is that a single checked basis can

classify *all* natural initial populations and identify exactly which extra resources permit a bad execution. Similarly, queue reasoning retains an unbounded amount of ordered data without requiring a finite queue-length cutoff. Neither feature requires the source user to supply the final invariant.

The new closure-separation producer also addresses an integration issue that a bare instruction to “use a Petri-net solver” would miss: a local predecessor basis can be enormous even when the existing global unsafe region covers it with one cone. The certificate language should be able to express the cheap global proof without materializing that intermediate basis.

1.3 Relationship to established research

Backward coverability based on well-quasi-orderings is classical, notably in the work of Abdulla, Čerāns, Jonsson and Tsay, and Finkel and Schnoebelen.[8, 9] The lossy-word predecessor operations below are single-letter specializations of established channel-machine operations.[10] This article does not rename those results as new mathematics. Its original deliverables are the selected Forge extension, explicit certificate formats, two working producers, a separately written checker, concrete witnesses, tests, and a port plan that preserves the exact meaning of the original Lean obligation.

2 One exact question over two infinite domains

2.1 Unsafe means existential finite reachability

Let S be a state space, T a finite set of transitions, and $\rightarrow_t$ the relation for transition t . Write $\rightarrow = \bigcup_{t \in T} \rightarrow_t$ and let $\rightarrow^*$ be its reflexive transitive closure. For a designated bad set,

$$\text{Unsafe} = \{s \in S : \exists z \in \text{Bad}, s \rightarrow^* z\}, \quad \text{Safe} = S \setminus \text{Unsafe}. \quad (1)$$

Thus $s \in \text{Safe}$ means that *every finite execution* from s avoids **Bad**. Membership in **Unsafe** means that *some* bad execution exists. It does not mean all executions fail, that failure is inevitable under a fair scheduler, or that a controller wins a game. No probabilities are involved.

The user may ask about one initial state, a symbolic family $I(\theta)$, or an initial predicate $\mathcal{I}$. The worker computes the same reusable **Unsafe** once. The remaining proof is respectively a finite membership test, a parameter formula, or

$$\forall s, \quad \mathcal{I}(s) \implies s \notin \text{Unsafe}. \quad (2)$$

2.2 Order and upward bases

A quasi-order $\preceq$ is reflexive and transitive. It is a *well-quasi-order* (WQO) if every infinite sequence $s_0, s_1, \dots$ contains $i < j$ with $s_i \preceq s_j$. For $B \subseteq S$, define

$$\uparrow B = \{s : \exists b \in B, b \preceq s\}.$$

We represent the bad set by a finite input list U with $\text{Bad} = \uparrow U$. For the two partial orders below, a finite irredundant basis is an antichain: no two distinct members are comparable.

States have a finite control component $q \in Q$. Order comparisons require *equal* controls; a state at an error location must not dominate a state at a different program point merely because its data are smaller. The data order is componentwise natural order for resources and componentwise subsequence order for queue contents.

Definition 2.1 (Strong upward compatibility). A transition relation is strongly upward compatible if

$$s \preceq s' \wedge s \rightarrow_t z \implies \exists z', s' \rightarrow_t z' \wedge z \preceq z'.$$

The same transition label is used on both sides.

Both supported languages satisfy this condition. Consequently the predecessor of an upward set is upward. The entire unsafe set is upward too: induct on an execution from the smaller state and lift each transition.

2.3 Why upward targets are an explicit restriction

“At least two processes are critical” is upward in resource counts. “The control is error” is upward in queue contents. “The final queue contains *ab* as a subsequence” is also upward. In contrast, an exact counter value, an exact queue word, or a queue being empty at an otherwise unrestricted control is not an upward property.

Replacing an exact target u by $\uparrow\{u\}$ changes the question from reachability to coverability. It can be a deliberately certified *over-approximation* for a safety proof, but it cannot silently count as an exact translation. In particular, representing a bad-control target by the empty word is valid because *all* queues at that control are bad, not because arbitrary empty-queue reachability has been solved.

3 A finite certificate for an exact infinite region

3.1 Two inclusions, with different evidence

The worker proposes a final basis B . To establish

$$\text{Unsafe} = \uparrow B, \tag{3}$$

it must establish both inclusions. These are logically independent.

For $\uparrow B \subseteq \text{Unsafe}$, it gives a finite acyclic derivation graph. A node is a concrete state, together with one of two reasons:

1. It is exactly one of the original bad basis states $u \in U$.
2. It names a transition t and an *earlier* node v , and its state s can take a t -step to some $z \succeq v$.

The final basis refers to nodes in this immutable graph. It is legal for a node to disappear from the active antichain while remaining in the derivation graph: later proofs may still depend on it.

For $\text{Unsafe} \subseteq \uparrow B$, it gives target coverage and backward closure:

$$\uparrow U \subseteq \uparrow B, \tag{4}$$

$$\text{Pre}_t(\uparrow\{b\}) \subseteq \uparrow B \quad \text{for every } b \in B \text{ and every } t \in T. \tag{5}$$

Control-incompatible pairs have empty predecessor sets and are discharged by checking their controls. The checker, not the producer, determines the full set of remaining transition/basis pairs.

Theorem 3.1 (Exact-region certificate soundness). *Assume strong upward compatibility, a finite bad basis, valid derivation nodes for every member of B , and (4)–(5). Then (3) holds. No WQO assumption is required for this theorem.*

Proof. First prove by induction on node index that every state above that node can reach **Bad**. At a bad leaf this follows from upward closure. At a predecessor node s , the recorded step reaches above the earlier child. From any $s' \succeq s$, compatibility lifts that step to another state above the same child; apply the induction hypothesis. Every state in $\uparrow B$ is above one of these nodes, proving the first inclusion.

For the second inclusion, induct backwards on the length of a concrete bad execution. Its final state lies in $\uparrow U \subseteq \uparrow B$. If $x \rightarrow_t y$ and $y \in \uparrow B$, choose $b \in B$ with $b \preceq y$. Then $x \in \text{Pre}_t(\uparrow\{b\})$, and the corresponding closure obligation puts x in $\uparrow B$. This reaches the initial state of every finite bad execution. $\square$

Remark 3.2 (Why the two halves must not be conflated). Backward closure alone accepts the whole state space and says nothing about whether its states really can fail. Derivations alone describe only discovered failures and cannot justify safety of an unrepresented state. Their combination proves exactness. A budget cutoff may still leave useful verified witnesses, but it does not manufacture the missing closure half.

3.2 The eager backward algorithm

For each principal cone, suppose an exact finite predecessor basis is computable:

$$\text{Pre}_t(\uparrow\{b\}) = \uparrow P_t(b).$$

The baseline producer uses the following worklist. It is included as an executable reference and an ablation baseline, not as a new coverability algorithm.[8, 9]

```

active := empty antichain
nodes  := immutable vector of derivations
queue  := empty
admit every original bad target

while queue is not empty:
  n := pop_front(queue)
  if n is no longer active: continue
  for each original transition ending at control(n):
    for p in exact_predecessor_basis(transition, state(n)):
      if some active b <= p: continue
      append node(p, transition, child=n)
      remove active states >= p, retaining their graph nodes
      activate the new node and enqueue it

rebuild all final transition/basis coverage obligations
return nodes, final active basis, target coverage, closure receipts

```

Theorem 3.3 (Termination and completeness of the idealized producer). *If S is WQO, every local predecessor computation terminates with an exact finite basis, and no operational cutoff is imposed, this worklist terminates and produces a certificate for (3).*

Proof. The upward set represented by the active antichain never shrinks. Removing an active v because a newly admitted $p \preceq v$ preserves all states it covered. If infinitely many states $s_0, s_1, \dots$ were admitted, WQO would give $i < j$ with $s_i \preceq s_j$. At admission time of s_j , the current upward region still contains s_i , and hence contains s_j . That contradicts the admission test. Thus only finitely many nodes are admitted. Each is popped once, each transition list is finite, and each local computation terminates, so the worklist finishes.

Every admitted node has the recorded valid derivation. Target coverage is preserved from initialization. At termination, every surviving node was expanded. Its predecessors were either admitted or already covered; later changes only enlarge the represented upward set. Therefore all final closure obligations hold, and [theorem 3.1](#) applies. $\square$

The actual program has limits on nodes, expansions and arithmetic partition work. Its completed finite receipt is checked without trusting this termination argument. Exhausting a limit returns `unknown`, not `safe` and not “the theorem is false.”

3.3 Minimality and finite witness plans

For our partial orders, exactness and antichain irredundancy imply that the final basis is precisely the set of minimal unsafe states. To see this, let $b \in B$ and suppose an unsafe $x \prec b$ existed. Exactness supplies $b' \in B$ with $b' \preceq x \prec b$, contradicting the antichain property. Conversely every minimal unsafe state lies above some basis member and must equal it.

Corollary 3.4 (A finite family of universal failure plans). *Each final basis node determines a finite action word that works from every state above that node, with suitable concrete states and, in the FIFO case, suitable losses. If the certificate contains N nodes, these words have length at most $N - 1$.*

Proof. Follow the strictly decreasing child indices to a bad leaf. Compatibility lifts the same transition labels from any larger starting state. A strictly decreasing chain of indices uses at most $N - 1$ edges. $\square$

This is a bound on the supplied witness family, not a claim that the extracted word is globally shortest. It is also not a strategy forcing failure against an adversary: it constructs one possible bad execution.

4 Worker I: guarded affine resources

Implementation status. Implemented by both Python producers; checked by a separately written arithmetic/partition replay.

4.1 The accepted transition language

Fix a finite Q and dimension d . States are (q, x) with $x \in \mathbb{N}^d$, ordered componentwise at equal control. A transition is

$$t = (q, q', a, A, b), \quad x \geq a, \quad x' = A(x - a) + b, \quad (6)$$

where $a, b \in \mathbb{N}^d$ and $A \in \mathbb{N}^{d \times d}$. All assignments are simultaneous and read the old state. The guard is explicit; it is not dropped because natural subtraction happens to be total in Lean.

Ordinary token transitions use $A = I$. A reset is a zero row; a transfer or merge puts several unit entries in one row; duplication permits entries larger than one. Different finite locations encode different control states. The prototype deliberately rejects negative entries, floating-point coefficients and unsupported fields, including zero tests.

Lemma 4.1 (Monotonicity). *Every transition (6) is strongly upward compatible.*

Proof. If $x \leq y$ and $x \geq a$, then $y \geq a$ and $y - a = (x - a) + (y - x)$. Since A is nonnegative, $A(y - a) + b = A(x - a) + b + A(y - x) \geq A(x - a) + b$. Both executions use the same transition and end at the same control. $\square$

Dickson's finite-product property makes $\mathbb{N}^d$ WQO. A finite union of its copies, with equality on the control component, is WQO as well. This supplies the termination hypothesis, but the replay checker needs only the local arithmetic statements in this section and the finite certificate theorem. The relevant finite-product and antichain formalization infrastructure is already present in Mathlib.[11]

4.2 Reducing an infinite preimage to a finite integer box

For a target vector u at q' , put

$$r_j = \max(u_j - b_j, 0), \quad z = x - a. \quad (7)$$

The exact predecessor condition is

$$x \geq a \quad \text{and} \quad Az \geq r. \quad (8)$$

For each input coordinate define

$$C_i = \max_{j:A_{ji}>0} \left\lceil \frac{r_j}{A_{ji}} \right\rceil, \quad \max \emptyset = 0. \quad (9)$$

Lemma 4.2 (Coordinate clipping). *If $z \in \mathbb{N}^d$ satisfies $Az \geq r$, then $\bar{z}_i = \min(z_i, C_i)$ also satisfies $A\bar{z} \geq r$. More generally, this holds for any natural C_i satisfying $A_{ji}C_i \geq r_j$ for every positive entry A_{ji} .*

Proof. Fix a row j . If no coordinate contributing positively to that row was clipped, its sum is unchanged. Otherwise choose one clipped coordinate i with $A_{ji} > 0$. Its contribution after clipping is $A_{ji}C_i \geq r_j$, already sufficient to meet the whole row demand. All other terms are nonnegative. Thus every row remains satisfied, even when several coordinates are clipped simultaneously. $\square$

Corollary 4.3 (Exact finite predecessor basis). *Let $F = \{z \in \mathbb{N}^d : 0 \leq z \leq C, Az \geq r\}$. Then*

$$P_t(u) = \{a + z : z \in \text{Min } F\} \quad \text{satisfies} \quad \text{Pre}_t(\uparrow\{(q', u)\}) = \uparrow\{(q, p) : p \in P_t(u)\}. \quad (10)$$

Proof. Every displayed predecessor is feasible. Conversely clip any feasible residual z into the finite box. Among feasible points below the clipped point, choose a componentwise minimal one. Adding a gives a listed predecessor below the original state. Minimal solutions cannot lie outside the box, since clipping would produce a strictly smaller feasible point. $\square$

This bound is elementary but particularly useful at the verification boundary: the checker verifies multiplications $A_{ji}C_i \geq r_j$. It need not repeat the producer's ceiling/max implementation. Zero rows with positive demand make the feasible set empty, which is also detected by the replay.

4.3 A cheap exact rule for diagonal maps

If A is diagonal, each coordinate is independent. For $A_{ii} > 0$ the least residual is $\lceil r_i/A_{ii} \rceil$. For $A_{ii} = 0$, a positive r_i makes the predecessor set empty, and $r_i = 0$ imposes no residual requirement. There is at most one minimal predecessor. This includes ordinary Petri transitions:

$$\text{pred}_t(u) = a + \max(u - b, 0) \quad (A = I). \quad (11)$$

The implementation takes this branch before considering any box enumeration. A huge numeric threshold therefore need not create a huge local search grid.

4.4 Independent finite-box replay

The eager producer enumerates F and discards dominated residual vectors. The checker does *not* rerun that enumeration. It receives the proposed minimal residuals, clipping bounds, and a binary integer-box partition. For an inclusive box $[L, H]$, the permitted leaves are:

Covered.

A proposed feasible residual m satisfies $m \leq L$. Every point in the box lies above it.

Impossible.

A row j satisfies $\sum_i A_{ji} H_i < r_j$. By nonnegativity, every point in the box violates that row.

Split.

For some i and $L_i \leq k < H_i$, split into the exact disjoint integer boxes obtained by replacing H_i by k and L_i by $k + 1$.

The checker also tests that the proposed residuals are feasible, in the box, sorted and pairwise incomparable. Coverage of the whole root box and clipping then establish [theorem 4.3](#). A redundant or malformed list cannot be accepted merely because its elements happen to be feasible.

The scheme is complete as a finite certificate language for these boxes. Repeated splitting reaches singletons. A feasible singleton is above a minimal feasible residual; an infeasible singleton violates some row. This is a mathematical completeness fact, not a promise that an enormous tree fits the program's resource limit.

4.5 Worked example: all semaphore populations at once

Write the resource state as (n, c, ℓ) : idle processes, critical processes, and available locks. The transitions are

$$\begin{aligned} \text{enter} &: (n, c, \ell) \mapsto (n - 1, c + 1, \ell - 1), \quad n, \ell \geq 1, \\ \text{leave} &: (n, c, \ell) \mapsto (n + 1, c - 1, \ell + 1), \quad c \geq 1. \end{aligned}$$

The bad target is $c \geq 2$, represented by $(0, 2, 0)$. The executed producer returns exactly

$$B = \{(0, 2, 0), (1, 1, 1), (2, 0, 2)\}. \quad (12)$$

The two additional states arise by taking predecessors of the bad target through one and two enters. Leave predecessors introduce no additional minimal unsafe states. The full certificate checks all six transition/basis pairs, not just those used to discover the three nodes.

In particular, $(N, 0, 1)$ is safe for *every* $N \in \mathbb{N}$: it dominates none of [\(12\)](#). The second and first vectors require $c \geq 1$ or $c \geq 2$, while the third requires two locks. The final arithmetic

proof can be discharged in Lean after the exact-region theorem has been imported; it contains no reachability reasoning.

A deliberately faulty enter returns the consumed lock immediately. With the same leave transition, the computed basis is

$$\{(0, 2, 0), (1, 1, 0), (2, 0, 1)\}.$$

The middle state really is unsafe even with no available lock: leave first creates a lock, after which two faulty enters reach the error. This is a useful example of why the final threshold set should be computed from *all* transitions rather than guessed from the visibly broken instruction. The prototype exports a concrete execution for each unsafe demonstration query.

4.6 Merging and amplification

For a transition $(x, y) \mapsto (x + y, 0)$ to an error-checking location with target $(4, 0)$, the source basis is

$$(0, 4), (1, 3), (2, 2), (3, 1), (4, 0).$$

Add a source self-loop $x \mapsto 2x$ guarded by $x \geq 1$, leaving y unchanged. The source unsafe basis shrinks to just

$$(0, 4), (1, 0).$$

The implementation retains ten derivation nodes while the final basis across both controls has three states. Deleting obsolete antichain members must not delete the historical reasons still referenced by the surviving nodes.

The one-dimensional fixture $x \mapsto 2x$ with the same guard and target $x \geq 10^{100}$ reaches basis $\{1\}$ after 334 admitted/expanded nodes. The extracted execution from $x = 1$ contains 333 doubling steps. The producer does not execute 10^{100} steps and makes no such claim: it uses exact large integers and logarithmically many backward thresholds.

5 Worker II: lossy FIFO channels

Implementation status. Implemented for a finite alphabet, a fixed finite number of channels, and single-symbol send/receive/tau transitions.

5.1 The order is subsequence, not prefix or substring

A channel contains a finite word over a finite alphabet Σ . Write $u \sqsubseteq w$ when u is obtained by deleting zero or more symbols from w , preserving order. Thus $ab \sqsubseteq aabb$ and $ab \sqsubseteq bab$, whereas $ab \not\sqsubseteq ba$. For k channels, compare each of their words independently at equal control. Higman's lemma and finite products make this order WQO. Mathlib's formalization uses `List.SublistForall2` over a base relation; specializing the base relation to equality gives the intended embedding order.[\[11\]](#)

The prototype's alphabet is a list of distinct single-character symbols. This is an implementation encoding of finite letters, not a license to erase arbitrary data payloads into characters. A Lean port should use a finite type such as `Fin alphabetSize`, together with a proved source encoding.

5.2 Explicit loss semantics

Each action permits a loss before the reliable action and a loss afterward. For one active channel, the equivalent direct relations are

$$w \xrightarrow{!a} v \iff v \sqsubseteq wa, \quad (13)$$

$$w \xrightarrow{?a} v \iff av \sqsubseteq w, \quad (14)$$

$$w \xrightarrow{\tau} v \iff v \sqsubseteq w. \quad (15)$$

Other channels may lose symbols but do not perform a send or receive during this action. These relations preserve FIFO order. A receive consumes a head symbol *after* possible prefix losses; it cannot reorder or invent messages. This agrees with standard lossy-channel semantics.[10]

Monotonicity is immediate: a larger input can first delete down to the smaller one and reproduce its execution. Extra pure loss steps need not be added to the transition list for upward-target coverability: the predecessor of an upward region under pure loss is that same region.

5.3 Exact principal predecessors

For a target word u , define

$$\rho_{!a}(u) = \begin{cases} v & u = va, \\ u & \text{otherwise,} \end{cases} \quad \rho_{?a}(u) = au, \quad \rho_{\tau}(u) = u. \quad (16)$$

Only the active channel changes; all other target words are preserved.

Lemma 5.1 (Single-symbol predecessor law). *For each of the three action kinds,*

$$\text{Pre}_{\theta}(\uparrow\{u\}) = \uparrow\{\rho_{\theta}(u)\}.$$

Proof. For receive, a possible post-state covering u exists exactly when $au \sqsubseteq w$. Necessity follows by composing the embeddings through the received state. Sufficiency deletes to au , consumes its first symbol, and leaves u .

For send, the condition is $u \sqsubseteq wa$. If an embedding uses the appended last position, that position must represent the last symbol of u , which must therefore be a . Removing it leaves the prefix v embedded in w . If $u = va$ but an embedding does not use the appended symbol, u and hence v already embed in w . Conversely $v \sqsubseteq w$ gives $va \sqsubseteq wa$. When u does not end in a , no embedding can use the appended position and the condition is simply $u \sqsubseteq w$. The empty word falls into this latter case and requires no input.

For tau, loss can leave a word covering u exactly when $u \sqsubseteq w$. The same argument applies componentwise to the unchanged channels. $\square$

Every local predecessor set therefore has a single minimal vector of words. The global antichain can nevertheless contain many incomparable vectors and can grow expensive. Neither Higman's lemma nor the one-predecessor rule gives a small uniform runtime bound.

5.4 A send loop and two ordered receives

Consider controls q_0, q_1, q_2 with q_2 bad and actions

$$q_0 \xrightarrow{!b} q_0, \quad q_0 \xrightarrow{?a} q_1, \quad q_1 \xrightarrow{?b} q_2.$$

The computed basis is

$$(q_0, a), \quad (q_1, b), \quad (q_2, \varepsilon).$$

Without the send loop the q_0 basis would require ab . With the loop, any queue containing an a as a subsequence can append a b , discard unneeded letters and perform the two receives. The certificate has four historical nodes because the earlier ab requirement is superseded by a . This is unbounded queue reasoning: the result applies to all queue lengths, not just the lengths used in the finite differential tests.

The two-channel fixture similarly records a requirement of an a in one channel and a b in the other, respecting the transition order through the finite controls. Componentwise comparisons never let surplus data on one channel satisfy a requirement on the other.

5.5 Lossy is not reliable

The model with one transition $q_0 \xrightarrow{?a} q_1$, with q_1 bad, is unsafe from queue ba under lossy semantics. Delete b , then receive a . Under reliable semantics that receive is blocked and there is no bad execution in this two-control example.

Consequently a lossy counterexample is not automatically a counterexample to reliable FIFO code. Reliable executions embed in lossy executions, so *safety* of the over-approximation can transfer to reliable code once the source bridge is proved. A reported lossy failure remains an abstract witness unless it passes a separate reliable replay. The provided `lossy_not_reliable` fixture makes this distinction observable.

6 Do not enumerate a local basis just to prove it is covered

Implementation status. A second producer and an additional independently checked receipt language are implemented and tested.

6.1 The avoidable intermediate object

The eager algorithm constructs $P_t(u)$ and then checks that each of its members lies in the current global region $\uparrow B$. But the necessary closure obligation is only

$$\text{Pre}_t(\uparrow\{u\}) \subseteq \uparrow B. \quad (17)$$

A full minimal predecessor basis is sufficient evidence, not logically necessary evidence. This difference can be enormous.

Suppose a transition consumes e_1 , merges d residual counters into one, and is asked to reach threshold h . Its minimal residuals satisfy $z_1 + \dots + z_d = h$, so there are

$$\binom{h + d - 1}{d - 1} \quad (18)$$

of them. If the current global source basis already contains e_1 , every predecessor is covered *solely by the guard*. Constructing all of (18) before checking coverage is wasted work.

6.2 Direct global-region partition receipts

Use the same clipping box $[0, C]$ as in [theorem 4.2](#). In place of local minimal residuals, permit a covered leaf to refer directly to a final global basis state v at the source control, and require

$$v \leq a + L. \quad (19)$$

Impossible and split leaves are unchanged. Coverage of the root box proves that every feasible residual in that box yields a state in $\uparrow B$. For an arbitrary feasible residual z , clipping gives $\bar{z} \leq z$ and a covered $a + \bar{z}$; upward closure then covers $a + z$. Thus the tree proves (17) directly.

The immutable derivation DAG still supplies the opposite inclusion. Omitting the local Pareto basis weakens neither the final exactness statement nor the counterexample authority. It merely removes an intermediate data structure that the upper-inclusion theorem did not require.

6.3 Closure separation as a search primitive

A local procedure now has two possible successful answers:

$$\text{Covered}(\text{partition proof}) \quad \text{or} \quad \text{Uncovered}(p, \text{concrete predecessor step}). \quad (20)$$

The latter point need not be a *locally* minimal predecessor. It only needs to be feasible and outside the currently represented upward region.

The implemented recursive box search follows this order:

```

probe(L, H):
  if some global basis v satisfies v <= consume + L:
    return a global-basis coverage leaf
  if some matrix row is below its demand at H:
    return an impossible leaf
  if L itself meets every row demand:
    return the uncovered state consume + L
  split the widest non-singleton coordinate at its midpoint
  recursively inspect both children

```

At a feasible singleton not covered by the region, the third branch must return a witness. At an infeasible singleton, the second branch must succeed. Therefore the procedure terminates on a finite box and returns either a sound coverage tree or an actual uncovered predecessor, absent cutoffs.

On receiving a witness, the producer admits its derivation node, updates the antichain, and probes again. It continues until the transition is covered, or until the target node itself is dominated and will be superseded by an enqueued smaller target. It processes every surviving target and every incoming transition. At the end it reconstructs all final coverage trees against the final basis indices: an old tree cannot be reused with stale references after the antichain changes.

Theorem 6.1 (Correctness of separation-based saturation). *With exact local separation, strong upward compatibility, a WQO, and no operational cutoffs, the separation-based worklist terminates and produces the same minimal unsafe basis as the eager worklist.*

Proof. Every admitted state is an uncovered predecessor of an existing derivation node, hence has a valid lower-inclusion reason and strictly enlarges the represented upward set. Infinite admission is excluded by the same bad-sequence argument as in [theorem 3.3](#). Once admissions stop, each remaining probe terminates with coverage. Surviving nodes receive all required coverage checks, giving (5); target coverage was preserved. Apply [theorem 3.1](#). Since the final basis is an antichain in a partial order, it is the unique set of minimal unsafe states, independently of which uncovered predecessors were chosen. $\square$

6.4 Executed ablation, with the exact claim

The two producers were run on the same 190 models and produced identical final bases. The separation checker independently replayed those 190 certificates and three additional dense-preimage fixtures. These are repeated models, not 380 distinct benchmark systems.

d	h	Local minima by (18)	Local grid size	Certificate bytes
4	20	1,771	194,481	776
6	60	8,259,888	51,520,374,361	898
8	100	26,075,972,546	10,828,567,056,280,801	1,063

Table 2: Constructed globally covered preimages. Minima/grid counts are exact formula evaluations, not objects enumerated during the experiment. Bytes count the full canonical compact-JSON region certificate.

Each fixture has two source-to-target transitions. The first consumes e_1 and produces at least the bad threshold, establishing the source basis e_1 . The second is the guarded dense merger described above. The eager producer returns **unknown** at its 100,000-point local-grid limit. The separation producer returns a complete certificate with two derivation nodes, two final basis states, and a one-leaf direct proof for the dense preimage. Its two counted partition visits are one during search and one during final certificate construction. No giant basis was generated and no giant trace was expanded.

This is an ablation of *our two Python implementations*, designed to expose an unnecessary intermediate representation. It is not evidence that existing optimized Petri-net tools fail on the fixtures, and it is not a comparison against Lean or **grind**.

6.5 Further concrete optimizations, not yet implemented

Factor the arithmetic constraint graph. Form a bipartite graph with one node for each residual variable, one for each positive-demand row, and an edge for each positive coefficient. Disconnected components involve disjoint variable sets. Handle isolated rows immediately: a positive-demand row with no incident variable is impossible. Solve each remaining component independently. The local feasible set is a Cartesian product; its minimal vectors are exactly products of componentwise minima. This can reduce arithmetic work greatly, although explicitly listing the product may still be exponentially large. A direct global-cover tree can avoid that product when global cones suffice.

Use an integer solver only to find uncovered points. An optional Z3 search can assert $z \geq 0$, $Az \geq r$, and

$$\bigwedge_{v \in B \text{ at source } i} \bigvee (a_i + z_i < v_i).$$

This is quantifier-free linear integer arithmetic because A is fixed. A satisfying model proposes an uncovered predecessor, which is independently checked. Coordinatewise minimization within that model's lower box can use successive bounded integer queries. The accepted local point need not be optimal. An **unsat** answer is *not* the closure proof: request or construct the direct partition receipt and replay it. Z3's documented `SolverFor("QF_LIA")`, integer terms and models provide a concrete backend for this optional search path.[13]

Index dominance without changing its meaning. Store resource states in a separate dominance index per control. A trie over coordinates with subtree lower/upper bounds can

reject whole subtrees when a prefix cannot dominate the query; a final exact componentwise comparison still decides admission. For words, length and letter-count vectors are cheap *necessary* embedding filters, followed by an exact subsequence test. Equal lengths or equal Parikh vectors are not sufficient for embedding. Caching these filters is an optimization, not a quotient of the state space.

7 What Forge should export after checking

7.1 An exact weakest safety precondition

For resource states at control q , the exact result is the finite formula

$$\text{Unsafe}(q, x) \iff \bigvee_{b \in B_q} \bigwedge_{i=1}^d (b_i \leq x_i), \quad (21)$$

$$\text{Safe}(q, x) \iff \bigwedge_{b \in B_q} \bigvee_{i=1}^d (x_i < b_i). \quad (22)$$

Empty disjunction and conjunction conventions give the correct answers for an empty bad set. A basis vector of all zeros makes its whole control unsafe. The meaning is the weakest initial condition for avoiding the specified bad set under every finite execution of the exact model.

For a symbolic family $x = I(\theta)$, substitute that expression in (22). For affine natural initializers the residual formula is Presburger arithmetic, so Forge can route it to its ordinary arithmetic layer. Polynomial or other initializers can use existing specialized workers without changing the region certificate. The new engine resolves the unbounded transition closure; the existing engine resolves the remaining initial-condition implication.

7.2 Executable witnesses, not just basis membership

An unsafe initial state is assigned to any basis node below it. Follow the node’s child links. For resource transitions, execute the original matrix update on the actual current state. Do *not* replace that state by the smaller recorded child: the larger values can affect subsequent execution. Monotonicity proves that the original action word remains enabled.

For lossy transitions, emit the exact pre-loss words, the result of the reliable action, and the post-loss words. The independent trace checker verifies each deletion direction, that receives consume the current head, that sends append at the end, and that the final state covers an original bad target. It does not trust a table of claimed embeddings or an output label saying “counterexample.”

The implementation offers two distinct artifacts: the region certificate and the concrete trace for a requested initial state. A valid trace can refute safety for that exact model and initial state even without a region certificate. A valid region certificate classifies every initial state even when no concrete trace has yet been requested.

8 A concrete Lean integration plan

Implementation status. Specified, not implemented or compiled in this work. The new Python checkers are not formally verified.

8.1 Keep the trust target small and semantic

Forge already specifies its acceptance boundary and has some elaborated core-only specimens; this work does not undo that progress or claim that nothing in Forge has compiled. What is missing *here* is a Lean implementation and soundness proof of these particular checkers, followed by an original-goal bridge.[2]

The first deliverable in Lean should be a *finite-certificate theorem*, not a formalization of a complicated terminating search. The search may remain in Python, or later move to a faster language. The checker needs no WQO theorem to justify an accepted finite certificate. WQO formalization can be a separate completeness milestone. This separation makes it possible to get a useful proof-producing worker without first formalizing all search scheduling, dominance indexing or solver internals.

Use Forge’s existing runtime and closure infrastructure, adding modules with roughly the following responsibilities. The names below are proposed project paths, not modules that currently exist.

Proposed module	New obligation it owns
<code>Forge/Ordered/Region.lean</code>	Upward regions, finite derivation DAG interpretation, complete transition/basis coverage, exact-region theorem.
<code>Forge/Ordered/Resource.lean</code>	Guarded nonnegative-affine semantics, strong monotonicity, residual conversion, clipping and diagonal predecessor lemmas.
<code>Forge/Ordered/BoxCover.lean</code>	Integer partition interpretation and both local-minimum and direct-global-cover soundness.
<code>Forge/Ordered/Lossy.lean</code>	Word/channel semantics, exact send/receive/tau predecessor laws and concrete loss replay.
<code>Forge/Ordered/Decode.lean</code>	Versioned data decoding into bounded, dimension-correct internal structures. No semantic authority in raw JSON.
<code>Forge/Ordered/Reify.lean</code>	Source-to-model equivalence or explicitly one-way simulation; original target correspondence.
<code>Forge/Ordered/Replay.lean</code>	Evaluate the checker, build its correctness application, return the exact original proposition.
<code>Forge/Ordered/Complete.lean</code>	Optional formalized WQO termination and equality of eager/separation mathematical algorithms.

8.2 A theorem dependency graph

The new statements should be developed in the following order.

1. Prove that a finite sequence of compatible transitions lifts upward. This is the only generic monotonicity lemma needed by DAG replay.
2. Prove DAG-node soundness by induction on the node index; prove backward closure soundness by induction on finite executions. Combine them into [theorem 3.1](#).
3. For resources, prove [theorem 4.1](#), the guarded residual identity, and [theorem 4.2](#). Then prove each partition constructor sound, and conclude each predecessor receipt sound.
4. For queues, prove the operational equivalences in (15), then [theorem 5.1](#), then the concrete pre/post-loss replay theorem.

5. Prove that the executable checker accepting data implies all semantic premises of the region theorem. Finally connect the decoded model to the source relation and bad predicate.

All numeric dimensions should be explicit in Lean. A resource vector can have type `Fin d → Nat`; a matrix can have type `Fin d → Fin d → Nat`; control values can have type `Fin qCount`. A word can be a `List (Fin alphabetSize)`. The executable storage may be arrays or lists for efficiency, but a length proof must connect it to this semantic interpretation. No out-of-range index should become an implicit default zero.

The final theorem should express equality or pointwise equivalence of the unsafe set, not just a Boolean classification for an unbound serialized state. The intended mathematical signature is

$$\text{check}(M, C) = \text{true} \implies \forall s, \quad (\exists z, \text{Reach}_M(s, z) \wedge \text{Bad}_M(z)) \iff s \in \uparrow B_C.$$

The exact Lean declaration names and syntax remain implementation work; this article does not ship an uncompiled `sorry`-filled file pretending to satisfy the contract.

8.3 Source-model bridges: three different permission levels

Let the Lean source relation be $\Rightarrow$ on X , its bad predicate be D , and an encoder be $E : X \rightarrow S$.

Exact encoding. For a representation equivalence, require correspondence of initial states, bad predicates, and transitions, with enough back-translation to reconstruct source executions. For a noninjective quotient, a bare existential equality of quotient edges is insufficient: abstract steps must lift from *every* represented concrete state where the result is claimed. Otherwise independently realizable abstract edges can form a spurious path.

One adequate contract has source forward simulation, target equivalence $D(x) \leftrightarrow \text{Bad}(E(x))$, and a step-lifting property

$$E(x) \rightarrow s' \implies \exists x', x \Rightarrow x' \wedge E(x') = s'.$$

With these properties, exact model classification transfers to the source. More flexible relational simulations are possible, but their lifting quantifiers must be written just as carefully.

Safety over-approximation. For a safety-only bridge it is enough that source steps simulate to model steps (or finite model paths), and $D(x) \Rightarrow \text{Bad}(E(x))$. Model safety then implies source safety. Model unsafety is only a candidate counterexample, because it may exploit extra behaviors. The reliable-to-lossy queue translation is the central example.

Witness-only under-approximation. A concretely replayed source execution establishes a source failure even when no exact or over-approximate source model is available. This gives no authority to declare unobserved states safe. The three entry points should remain separate in the API, rather than sharing a Boolean `verified` flag. Forge’s existing evidence-boundary design supplies this discipline; these are its new, model-specific proof obligations.[4]

8.4 An implementable resource reifier

Start with an explicit-model entrance rather than silently analyzing arbitrary Lean code. The user supplies a finite transition list and a theorem connecting it to the source relation. This immediately supports serious proofs and isolates the checker work from general program extraction.

The next entrance can recognize inductively defined finite transition relations whose rules normalize to lower-bound guards and simultaneous nonnegative-affine updates. It should enumerate every constructor, assign a stable source reference, extract a, A, b , and prove the transition equation under the original guard. Polynomial normalization can help establish the coefficient identity, but ordinary natural subtraction must be rewritten only under the relevant lower-bound hypotheses. If a rule has an upper-bound guard, a zero test, a negative coefficient, an unbounded control variable or a value-dependent matrix, refuse the exact entrance rather than erase it.

A source expression such as $2x - 2$ is not globally interchangeable over naturals with an unconstrained integer expression. Under $x \geq 1$ it equals $2(x - 1)$, and that conditional equality is what the model bridge must prove. Likewise a sequence of assignments must first be converted to a simultaneous old-state expression; using the matrix of the written right-hand sides before substitution changes the program.

For anonymous finite-state populations, a counting quotient is especially valuable. Concrete states are multisets of agents; each transition consumes and produces specified multiplicities. A sufficient-count theorem chooses the required distinct agents and proves that every abstract step lifts. This justifies exact classification for every population size. Identity-sensitive protocols, nonexchangeable payloads and hidden local state need a stronger quotient proof or a safety-only abstraction, not an automatic exact label.

8.5 An implementable queue reifier

The initial queue entrance should accept an explicit finite-control relation over lists with single-symbol sends/receives and explicit lossy semantics. Provide the reliable-to-lossy safety bridge as a separate theorem. Enumerate and check every channel, control, symbol and action; an omitted action would make the computed safe set unsound for the original model.

For data-carrying messages, a finite alphabet map needs a proved behavioral contract. A many-to-one map may be an over-approximation, but it need not support counterexample transfer. The original source's equality tests, branching on payloads and typeclass dictionaries must remain accounted for. In particular, this worker does not solve the Church-indexing synthesis gap reported in the current Leant/Djex diagnostics. Applying subsequence pruning to arbitrary synthesized functions would be unsound without the very representation and behavior theorems that those diagnostics carefully keep separate.[5, 6, 7]

8.6 Concrete libraries and their roles

Mathlib's `Mathlib/Order/WellFoundedSet.lean` contains the finite-product and antichain infrastructure, including `Set.PartiallyWellOrderedOn.pi` and `IsAntichain.finite_of_partiallyWellOrderedOn`. Its documented Higman entry point is `Set.PartiallyWellOrderedOn.partiallyWellOrderedOn_sublistForall`. `Mathlib/Order/WellQuasiOrder.lean` supplies the underlying WQO vocabulary. These names were checked against the online documentation; the exact imports and elaboration at Forge's pinned dependency still need a compiler run. The pipeline must not infer that a documentation lookup is an API acceptance test.[11, 12]

Forge's reviewed baseline specifies Lean 4.34.0 and Mathlib commit `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa`. Use that pair for the first integration branch instead of mixing the repository with a moving Mathlib head.[1] TAPAAL and LoLA are concrete external verification projects, not certified Lean backends out of the box. Their adapters are optional future work.[14, 15]

Library/tool	Proposed use	Trust/implementation status
Lean 4 and Mathlib	Semantic statements, finite induction, integer arithmetic, lists, WQO infrastructure	Required for kernel acceptance; port not executed here
Python standard library	Exact integer prototypes, JSON receipts, deletion-mask oracles	Executed; no third-party runtime dependency
Z3 integer API	Optional uncovered-point proposal and local minimization	Not integrated or used by the delivered producers; solver answers are not receipts
TAPAAL / verifypn	Optional untimed token-net corpus and external producer adapter	Not integrated; admit only a proved supported subset, not arbitrary timed/game/inhibitor semantics
LoLA	Optional external token-net comparison and concrete trace source	Not integrated; independent source/-model replay still required

Table 4: Specific tools with explicit boundaries, rather than a list of solvers presumed to produce Lean proofs.

9 Executed experiments and what they establish

9.1 Environment and reproducibility

The recorded run used Python 3.13.5 on x86_64 Linux and seed 941572603. All delivered search and replay code runs under `python -S`, disabling site packages. The test corpus, raw accepted certificates, concrete illustrative traces, summaries and run output are included. Timings are single-run wall-clock observations, stored for diagnosis, not controlled performance estimates.

The corpus generator, explicit oracle and arithmetic checker are separate functions/modules. The checker imports neither producer. In a fresh replay process, an import hook makes attempts to import the search module or NumPy/SciPy/SymPy raise an exception. This verifies a real execution-path separation. It does not prove that independently written routines lack a shared mathematical misunderstanding.

Experiment and unit	Cases/queries	Discrepancies
Local resource predecessor problems	1,296	0
Resource-point membership comparisons on those problems	63,504	0
FIFO action/target/input word comparisons	2,325	0
Complete finite-resource execution queries, 100 systems	7,000	0
Complete acyclic FIFO execution queries, 40 systems	2,040	0
Complete length-nonincreasing FIFO queries, 40 systems	2,480	0
Eager region certificates independently replayed	190	0 rejected
Same-model separation/eager basis comparisons	190	0
Separation region certificates independently replayed	193	0 rejected
Eager/general parser and semantic corruption attempts	32	0 accepted invalid
Additional direct-cover corruption attempts	5	0 accepted invalid
Operational-cutoff controls	4	0 promoted to a verdict

Table 5: Distinct units remain distinct. Several rows intentionally reuse the same models; there is no combined success-rate denominator.

9.2 Local predecessor tests

For the resource lane, the generator covers every 2×2 matrix with entries in $\{0, 1, 2\}$ and every target in $\{0, 1, 2, 3\}^2$: $81 \cdot 16 = 1296$ problems. Consumption/production vectors are chosen deterministically by the recorded seed. On each problem, it compares the represented predecessor set with direct forward arithmetic for all 49 points of $\{0, \dots, 6\}^2$. This includes points beyond the clipping bounds and points below the guards. The finite input family is not exhaustive over arbitrary dimensions, coefficients or guards.

For FIFO, there are five concrete action variants: send/receive either binary letter, plus tau. Every binary input word of length at most four is compared against every target word of length at most three. The oracle enumerates actual deletion masks before and after the reliable action. It does not call (16), the producer’s greedy subsequence predicate, or the checker’s dynamic-programming embedding predicate. The $5 \cdot 31 \cdot 15 = 2325$ comparisons all agree.

9.3 Why the finite execution oracles really terminate

The resource systems have two controls and three counters. Every matrix column has sum at most one, and total production is at most total consumption. For an enabled transition,

$$\|A(x - a) + b\|_1 \leq \|x - a\|_1 + \|b\|_1 \leq \|x\|_1.$$

Each queried initial state has total resource at most four. The full reachable space lies in a finite simplex, so explicit BFS exhausts the actual reachable states rather than truncating an infinite search. The 7,000 queries divide into 910 unsafe and 6,090 safe results, with exact agreement.

For the acyclic channel family, finite control paths bound the number of sends; initial total length is at most two and there are two channels. For the cyclic family there are no sends, so total length never increases. These are independent reasons why explicit execution terminates. The acyclic queries divide into 124 unsafe and 1,916 safe; the decreasing-length queries into 1,319 unsafe and 1,161 safe. The oracle has a defensive state limit, but hitting it would raise an inconclusive test failure; it never becomes a safety answer. No query in the recorded run reached that limit.

The unbounded send-loop and amplification examples are covered by their finite certificate proofs, not by claiming that short explicit tests enumerate all their executions. This distinction is important even when every displayed test is green.

9.4 Corruption and refusal controls

The tests remove required closure rows and target coverage; duplicate rows and basis entries; create a cyclic child reference; corrupt a lower-inclusion state; tamper with clipping bounds, minima and partition nodes; incorrectly select the diagonal rule; invent messages during pre/post losses; and try invalid numeric values and duplicate JSON keys. All these intentionally invalid objects are rejected.

A source transition is also changed and the certificate subject deliberately rebound to that changed model. Rejection in this case depends on the semantic checks, not merely a mismatching subject string. Unsupported zero-test fields are rejected rather than ignored. The additional separation tests corrupt its direct global-basis references, partition and row completeness.

The four producer cutoff controls independently exhaust the node, expansion, local-grid and box-certificate budgets. All return **unknown**. These tests exercise selected failure paths; they do not establish resistance against hostile or arbitrarily large input. The decoder has explicit byte, dimension, word-length, item-count and integer-bit limits, but it is research code rather than a security-audited sandbox.

9.5 A preserved failed run

The first experiment run reached the named-example assertions and failed because the reporting dictionary used the key `basis` both for the list of basis states and for a numeric statistic. Merging the statistics replaced the list with its length. The produced semaphore certificate already contained the expected three vectors. The repair nested the statistics under `search_stats`; the experiment then ran again. The original traceback and run output are retained in `results/failed-runs/`. This was a reporting/test-harness defect, not evidence of a false region certificate; its scope should not be exaggerated in either direction.

9.6 Unmeasured claims

No theorem in this new package was checked by Lean. No `forge` tactic was installed. No Mathlib-dependent source was compiled. No benchmark against `grind`, `Aesop`, `omega`, `nlinarith`, `TAPAAAL`, `LoLA` or an external SMT solver was executed. The old repository’s test counts are not added to these. The article’s mathematical proofs, the Python results and a future Lean-kernel acceptance are three distinct levels of evidence.

10 Implementation priorities and acceptance gates

10.1 First merge: a bounded, explicitly invoked worker

The first useful integration should accept an explicit resource model and produce the exact-region theorem for diagonal transitions. This covers ordinary token systems, resets on independent coordinates and guarded amplification, while requiring no general box proof. Acceptance requires a compiled soundness theorem, successful replay of the original certificate against the pinned Lean/Mathlib pair, an original-source semantic bridge, and rejection of the deliberately faulty version. A theorem about a hand-rewritten model is not enough unless its equivalence to the source is also proved.

Next add the resource clipping lemma and direct-global-cover checker. This is more useful than first formalizing exhaustive local Pareto enumeration: it admits both producers’ useful results while avoiding the representation explosion demonstrated in [table 2](#). Keep the eager local-minimum receipt as a second checker path and an independent test fixture.

Then add the lossy-channel semantics and predecessor laws. The first acceptance corpus should include the send-loop example, the two-channel example, a correct concrete loss trace, and the reliable/lossy distinction. A test suite that validates only positive lossy examples has not checked the source abstraction boundary.

10.2 Required regression tests for the Lean port

Beyond porting the stored arithmetic and word cases, add exact source tests for simultaneous versus sequential updates, guarded natural subtraction, missing transition constructors, control reindexing, mismatched channel identities and many-to-one payload encodings. Test an upward target and a nearby exact target together, ensuring the latter is refused or carries an explicit approximation theorem.

For certificate construction, check the original goal in a clean verification context according to Forge’s existing policy. Check the axiom dependencies of the resulting declarations and the source bridge. A theorem that assumes `checkSound` as an axiom is not a certified checker. An

elaborated structure containing fields named `sound` is not a proof that any serialized input satisfies those fields.

For performance evaluation, separate source reification, producer time, certificate size, Lean replay time, and final residual-goal solving. Compare with baseline tactics on the same original source statements and with the same provided lemmas. Do not count a difficult model lemma as free preparation for one side while charging its proof to the other. A worker may be valuable even when its examples also have short manual invariants; the principal new metric is coverage of whole parametric initial-state families without supplied invariants.

10.3 Boundaries that should remain visible

The resource lane does not support zero tests or arbitrary negative updates. The queue lane does not decide reliable FIFO reachability, exact-word targets, infinite alphabets or fairness. The current implementation does not combine resource counters and queues in one model, although a product-domain worker with appropriate independent action semantics is a plausible next extension. It does not solve mixed games or probabilistic reachability by changing the interpretation of “unsafe.” Those subjects already have different workers and proof contracts in Forge.

WQO guarantees eventual termination of the idealized supported algorithms, not practical runtime. Large antichains and long bad sequences remain genuine costs. The direct-cover language avoids some intermediate explosions; it does not remove all output-size or state-space complexity. Timeout, memory refusal, unsupported reification and lack of a source bridge remain legitimate, explicit nonacceptance outcomes.

11 Conclusion

Forge should gain the ability to return an exact finite unsafe-region basis for two new infinite-state languages: guarded nonnegative-affine resources and lossy FIFO channels. The same result yields parameterized safety conditions, minimal resource/message requirements for failure, and a finite family of concrete action plans for every unsafe initial state.

The proposal is implementable in stages because finite certificate soundness does not depend on verifying the search’s WQO termination proof. Its most important representation decision is to let a checker accept direct coverage by the final global antichain rather than insist on enumerating every local minimal predecessor. The executed separation experiment illustrates the consequence with a 1,063-byte certificate where the unnecessary local predecessor basis has over 26 billion elements.

What exists now is a detailed semantic design, proved mathematical reductions, two executed Python producers, an independent finite-data checker, replayable witnesses and a reproducible test archive. What remains is the Lean checker and the original-source bridge. Those are precise engineering and formalization tasks, not gaps to be hidden behind a claim that the prototype already is a Lean tactic.

A Certificate format and executable entry points

A.1 Region object

The versioned JSON object has the following top-level fields:

```
{
  "format": "forge-ordered-v1",
  "subject": "canonical encoding of the complete original model",
  "nodes": [ ... immutable earlier-child derivations ... ],
  "basis": [ ... node indices in canonical state order ... ],
  "target_cover": [ ... one basis index per original bad target ... ],
  "closure": [
    { "basis": 0,
      "transition": 2,
      "predecessor": { ... receipt ... },
      "cover": [ ... basis indices ... ] }
  ]
}
```

The subject is the complete canonical model encoding, not a short hash. The checker receives the expected model separately and compares this encoding. The check is not a source-goal fingerprint: that extra relationship belongs to the future Lean reifier and source bridge.

Each derivation is either an exact original-target leaf or a state with a transition index and strictly smaller child index. The checker rejects unknown fields and out-of-range indices. Sorted states use control then lexicographic data order solely for deterministic encoding; semantic dominance is the separate componentwise or subsequence partial order.

A.2 Local receipt tags

The **diagonal** tag requires an actually diagonal matrix and uses exact integer quotient/remainder rules. The **box** tag contains natural clipping caps, feasible minimal residuals and a local-minimum partition. The **lossy-word** tag triggers the proved send/receive/tau predecessor rule. For these three tags, the closure row carries one coverage index per computed predecessor.

The **resource-cover** tag contains clipping caps and a partition whose covered leaves refer directly to final global basis indices. Its row carries an empty local **cover** list. All incoming transition/basis pairs must still occur exactly once. This direct tag does not bypass the lower-inclusion DAG or original-target coverage checks.

A future schema revision should preserve explicit version negotiation. It should not let an unknown receipt tag fall through to a default semantics, or let a malformed basis index mean the zero vector.

A.3 Commands

From the archive root:

```
# Reproduce the full first suite into a new directory.
python -S prototype/run_tests.py --output reproduced-results

# Compare the second producer on the same generated models.
python -S prototype/run_separation.py \
  --source reproduced-results --output reproduced-separation

# Independently check a stored example; no search is invoked.
python -S prototype/checker.py \
  results/examples/semaphore.problem.json \
  results/examples/semaphore.certificate.json

# Search and independently check a new explicit model.
```



```
python -S prototype/cli.py \
  results/examples/semaphore.problem.json --out new-region.json

# Replay the largest dense-preimage receipt.
python -S prototype/checker.py \
  results/separation/dense-8.problem.json \
  results/separation/dense-8.certificate.json
```

The CLI defaults to the separation producer. Its successful output explicitly says `accepted_by_python_checker` and `lean_kernel_checked: false`. It refuses to overwrite an existing certificate path. Optional initial-state input exports an independently checked concrete trace when the state is unsafe.

To rebuild this article, run `pdflatex` on `article/forged_ordered.tex` from its directory three times. No external bibliography processor, shell escape, network connection, or third-party Python library is required by the artifact.

B Evidence inventory

Artifact	Meaning
<code>prototype/search.py</code>	Both untrusted producers; exact arithmetic, subsequence search, witness extraction.
<code>prototype/checker.py</code>	Independently written region and concrete-trace replay; no producer import.
<code>prototype/run_tests.py</code>	Deterministic local tests, complete finite oracles, corruption and cutoff controls.
<code>prototype/run_separation.py</code>	Same-model producer ablation and dense-preimage fixtures.
<code>prototype/cli.py</code>	Explicit-model command line; checker acceptance required before a success output.
<code>results/summary.json</code>	Environment, grouped counts, named basis states, queries and raw single-run timing observations.
<code>results/corpus.json</code>	190 model/certificate pairs from the eager run.
<code>results/trace-corpus.json</code>	18 illustrative concrete traces from named examples; not all oracle witness checks are stored here.
<code>results/separation/corpus.json</code>	193 model/certificate pairs: the same 190 models plus three dense-preimage fixtures.
<code>results/separation/summary.json</code>	Exact-basis agreement, constructed-case size calculations and direct-cover mutation results.
<code>results/failed-runs/</code>	Preserved first reporting-harness failure and its original output.
<code>docs/novelty-audit.md</code>	The review boundary, reused ideas, delta, and non-exhaustiveness caveat.
<code>docs/LEAN_PORT_PLAN.md</code>	Missing formalization work and explicit acceptance conditions.

References

- [1] V. Reshetnikov, *Forge*, README at commit 58ea206bd7ad501b930add568151217bcc7f82a2. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/README.md>.
- [2] V. Reshetnikov, *Forge implementation status*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/STATUS.md>.
- [3] V. Reshetnikov, *Finite certificates for unbounded behaviour*, Forge merged article, Section 6 source, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/06-closure.tex>.
- [4] V. Reshetnikov, *Ideas from Leant and Djex*, Forge, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] V. Reshetnikov, *Leant*, README at commit 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [6] V. Reshetnikov, *Djex*, commit record e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, “Locate the remaining indexing gap in bounded composition search.” <https://github.com/VladimirReshetnikov/Djex/commit/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5>.
- [7] V. Reshetnikov, *Indexing composition after native kind adoption*, 14 September 2026, Leant diagnostic report at the reviewed snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [8] P. A. Abdulla, K. Čerāns, B. Jonsson and Y.-K. Tsay, “Algorithmic analysis of programs with well quasi-ordered domains,” *Information and Computation* 160(1–2), 109–127, 2000. <https://doi.org/10.1006/inco.1999.2843>.
- [9] A. Finkel and P. Schnoebelen, “Well-structured transition systems everywhere!,” *Theoretical Computer Science* 256(1–2), 63–92, 2001. [https://doi.org/10.1016/S0304-3975\(00\)00102-X](https://doi.org/10.1016/S0304-3975(00)00102-X).
- [10] P. Schnoebelen, *On flat lossy channel machines*, arXiv:2007.05269v1, 2020. In particular, Sections 2 and 3.1 for loss semantics and predecessors. <https://arxiv.org/html/2007.05269v1>.
- [11] The Mathlib community, *Mathlib.Order.WellFoundedSet*, online API documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Order/WellFoundedSet.html.
- [12] The Mathlib community, *Mathlib.Order.WellQuasiOrder*, online API documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Order/WellQuasiOrder.html.
- [13] The Z3 project, *Python API reference*, integer expressions, solvers, models and SolverFor. <https://z3prover.github.io/api/html/z3.z3.html>.
- [14] The TAPAAL project, official project site and verification tools. <https://www.tapaal.net/>. <https://github.com/TAPAAL/verifypn>.
- [15] The LoLA project, official tool page, University of Rostock. <https://theo.informatik.uni-rostock.de/en/theo-forschung/tools/lola/>.